

Chandpur Science and Technology University



Department of Computer Science and Engineering

Course Code: CSE 3202

Course Title: Operating System Sessional

Lab Manual 2

Lab Title: Introduction to Linux with WSL (Windows Subsystem for LINUX).

1. Why Learn Shell Scripting?

When we use a computer, we usually interact through a GUI — clicking icons with a mouse. But in the real world, most servers have no GUI at all. Everything must be done through the Command Line or Shell. This is why shell scripting is one of the most essential skills for any Computer Science student.

1.1 Real-World Usage

- Web servers, database servers, and cloud servers all run on Linux
- Companies like Netflix, Google, and Amazon rely entirely on Linux shell for server operations
- A single shell script can process thousands of files automatically
- Automated backups, deployments, and monitoring all use shell scripting

2. What is a Shell and How Does it Work?

A shell is a program that reads your commands and tells the Operating System what to do. When you type something in the terminal, the shell interprets it and asks the kernel to execute it. Think of the shell as a translator between you and the OS.

2.1 Types of Shells

Shell	Full Name	Notes
sh	Bourne Shell	Oldest, most basic
bash	Bourne Again Shell	Most popular — we will use this
zsh	Z Shell	Default on macOS, advanced features
fish	Friendly Interactive Shell	Beginner friendly, colorful output

3. Installing WSL in your windows operating system

You can follow this video: <https://www.youtube.com/watch?v=XGBsmrUdwkk>

3.1. Understanding the Shell Prompt

When the terminal opens, you will see something like this:

```
username@hostname:~$
```

Command	What It Does
username	This is your Linux username — who you are logged in as.
hostname	The name of your computer (your machine's identity on a network).
~	This symbol means your home directory — your personal folder. It is short for /home/username.
\$	This dollar sign means you are a normal user. If you see a # symbol instead, it means you are the root (admin) user.

4. Basic Shell Commands

These commands are the building blocks of shell scripting. You must know these before writing any script.

4.1 File & Directory Commands

Command	What it Does	Example
pwd	Show current directory	pwd
ls	List files	ls -la
cd	Change directory	cd /home
mkdir	Create a new folder	mkdir myFolder
touch	Create a new empty file	touch file.txt
rm	Delete a file	rm file.txt
cp	Copy a file	cp a.txt b.txt
mv	Move or rename a file	mv old.txt new.txt
cat	Display file contents	cat file.txt
echo	Print text to screen	echo "Hello World"

4.2 Practicing the commands.

1. Navigate to the **C:** drive.
2. Display all files and directories in the current location.
3. Create a new folder on the **Desktop**.
4. Create a new file on the **Desktop**.
5. Rename the newly created file and display its contents.

5. Your First Shell Script.

```
# This is my first shell script
echo "My username: $USER"
echo "Today's date: $(date)"
echo "Current directory: $(pwd)"
```

5.1 Line-by-Line Explanation

Line	What it Means
# comment	Anything after # is a comment. It is not executed.
echo	Prints text to the screen
\$USER	An environment variable holding the current username
\$(date)	Command substitution — inserts the output of the date command

6. Variables and User Input

5.1 Using Variables

```
name="John"      # string variable
age=20           # integer variable
echo "Name: $name"
echo "Age: $age years"
echo "Age in 5 years: $((age + 5))"
```

5.2 Taking Input from the User

```
echo "What is your name?"
read userName
echo "What is your roll number?"
read roll
echo "Welcome, $userName! Your roll: $roll"
```

5.3 Important Rules

- No spaces around = when assigning: name="John" is correct, name = "John" is wrong
- Always use \$ to access a variable's value: \$name
- Use \$(()) for arithmetic: \$((5 + 3))
- Use double quotes to embed variables in strings: "Hello \$name"

6. Conditional Statements (if-else)

6.1 Basic if-else

```
echo "Enter a number:"
read num

if [ $num -gt 0 ]; then
    echo "$num is a positive number"
elif [ $num -lt 0 ]; then
    echo "$num is a negative number"
else
    echo "The number is zero"
fi
```

6.2 Comparison Operators

Operator	Meaning	Example
-eq	Equal to	[\$a -eq \$b]
-ne	Not equal to	[\$a -ne \$b]
-gt	Greater than	[\$a -gt \$b]
-lt	Less than	[\$a -lt \$b]
-ge	Greater than or equal	[\$a -ge \$b]
-le	Less than or equal	[\$a -le \$b]

7. Loops

7.1 For Loop

```
# Print numbers 1 to 5
for i in 1 2 3 4 5; do
    echo "Number: $i"
done
```

```
# Using a range
for i in {1..10}; do
    echo "$i"
done
```

7.2 While Loop

```
count=1
while [ $count -le 5 ]; do
    echo "Count: $count"
    count=$((count + 1))
done
```

8. Lab Exercises

Exercise 1

Write a script that:

- Takes the user's name as input
- Takes the user's age as input
- Prints: "Name: [name], Age: [age], Birth Year: [calculated year]"

Exercise 2

Write a script that:

- Takes two numbers as input
- Tells which one is greater (or if they are equal)
- Prints the sum, difference, product, and quotient of the two numbers

Exercise 3

Write a script that:

- Loops from 1 to 20
- Prints "Even" if the number is even
- Prints "Odd" if the number is odd
- Additionally prints "Multiple of 3" if the number is divisible by 3